

Лабораторная работа №6

«Работа с базами данных»

Цель работы: Подключение и настройка к проекту библиотеки SQLAlchemy и библиотеки alembic. Создание реляционной базы данных при помощи библиотеки SQLAlchemy и реализация миграций с помощью библиотеки alembic.

Ход работы:

Для начала работы с указанными библиотеками необходимо их сначала установить в проект. Для этого откройте терминал в директории, где находится проект и выполните команду:

```
pip install SQLAlchemy alembic
```

После чего должны установиться необходимые зависимости.

Согласно официальной документации (документация библиотеки SQLAlchemy находится по URL: <https://flask-docs.readthedocs.io/en/latest/patterns/sqlalchemy/>), реализация задачи создания базы данных с помощью библиотеки SQLAlchemy включает в себя создание файла с подключением к базе данных (рис. 2), а также создания самой модели реляционной базы данных. Последняя создается с использованием соответствующих классов библиотеки.

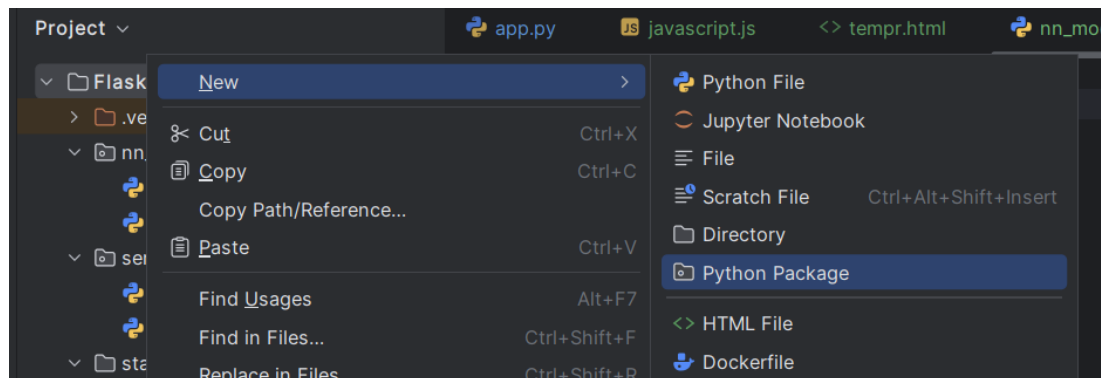


Рисунок 1. – Создание пакета с файлом подключения к БД.

Для подключения к базе данных создайте новый пакет (рис. 1), назовите его например «database». Внутри пакета создаем файл, где будет осуществляться соединение с базой данных. Назовем его, например database.py. В этот файл затем помещается программный код из документации (рис. 2)

```

from sqlalchemy import create_engine
from sqlalchemy.orm import scoped_session, sessionmaker
from sqlalchemy.ext.declarative import declarative_base

engine = create_engine('sqlite:///predict.db')
db_session = scoped_session(sessionmaker(autocommit=False,
                                         autoflush=False,
                                         bind=engine))

Base = declarative_base()
Base.query = db_session.query_property()

def init_db():
    """ 2 usages new *
    # import all modules here that might define models so that
    # they will be registered properly on the metadata. Otherwise
    # you will have to import them first before calling init_db()
    from db_model.results import Predictions # Импортируем модель
    #import models.user_model
    Base.metadata.create_all(bind=engine)
    """

```

Рисунок 2. – Код подключения к БД реализованный в файле «database.py»

В примере выше используется диалект SQLite. Подключение осуществляется в строке кода:

```
engine = create_engine('sqlite:///predict.db')
```

Здесь, в метод `create_engine`, необходимо передать строку подключения. В случае с SQLite строка подключения начинается с `sqlite:///`

Для других диалектов языка SQL эта строка будет отличаться. Далее указывается путь к файлу с таблицами SQL относительно текущего файла, в котором реализуется подключение (рис. 2). Назвать файл можно любым именем (в примере выше он назван `master`), важно, чтобы файл имел расширение `db`.

Следующим этапом необходимо реализовать саму модель баз данных с помощью классов SQLAlchemy. В данной работе она будет включать лишь одну таблицу, назовем ее например «Predictions». Поля (переменные класса) в данном классе должны соответствовать данным, которые необходимо хранить. Такими данными будут уникальное ID предсказания, название файла с изображением, само изображение, преобразованное в текст, предсказанный класс, уверенность предсказания, вероятность того, что изображение является окружностью, вероятность того, что изображение является треугольником, вероятность того, что изображение является квадратом и дата проведения классификации. Для всех этих данных необходимо прописать соответствующие переменные класса и реализовать конструктор (инициализатор).

Аналогично, создаем пакет для модели. Называем его, к примеру, «`db_model`». Внутри данного пакета необходимо создать файл, где будет находиться код реализации

модели баз данных. Назовите его results.py. Код его возможной реализации представлен на рисунке 3.

```
from datetime import datetime, timezone
import base64
from sqlalchemy.orm import declarative_base
from sqlalchemy import Column, Integer, String, Float, Text, DateTime

Base = declarative_base()

class Predictions(Base):
    __tablename__ = 'Predictions'

    id = Column(Integer, primary_key=True)
    filename = Column(String(255), nullable=True) # Оригинальное имя файла
    image_data = Column(Text, nullable=False) # Изображение в base64
    predicted_class = Column(String(50), nullable=False) # Предсказанный
    класс
    confidence = Column(Float, nullable=False) # Уверенность
    probabilities_circle = Column(Float, nullable=True) # Вероятность круга
    probabilities_square = Column(Float, nullable=True) # Вероятность
    квадрата
    probabilities_triangle = Column(Float, nullable=True) # Вероятность
    треугольника
    created_at = Column(DateTime, default=lambda: datetime.now(timezone.utc))
    # Время создания

    def __init__(self, filename, image_data, predicted_class, confidence,
                  probabilities_circle, probabilities_square,
                  probabilities_triangle):
        """
        Прямой инициализатор с явными параметрами
        """
        self.filename = filename
        self.image_data = image_data
        self.predicted_class = predicted_class
        self.confidence = confidence
        self.probabilities_circle = probabilities_circle
        self.probabilities_square = probabilities_square
        self.probabilities_triangle = probabilities_triangle

    def __repr__(self):
        """Строковое представление объекта"""
        return (f"<Prediction(id={self.id}, filename='{self.filename}',"
                f" class='{self.predicted_class}',"
                f" confidence={self.confidence:.2%})>")

    @staticmethod
    def image_to_base64(image_bytes):
        """Конвертирует изображение в base64 строку"""
        return base64.b64encode(image_bytes).decode('utf-8')

    @staticmethod
    def base64_to_image(base64_str):
        """Конвертирует base64 строку обратно в байты"""
        return base64.b64decode(base64_str.encode('utf-8'))
```

Рисунок 3. – Реализация модели БД в файле results.py

Реализация модели включает в себя необходимые импорты, создание класса Base следующей строчкой кода:

```
Base = declarative_base()
```

Каждая таблица баз данных корреспондируется с соответствующим классом. Переменные классов (поля) создаются при помощи соответствующих классов из библиотеки SQLAlchemy (Column, Integer, String и т.д.). Обязательным условием является наследование классов от класса Base, а также метода приведенного ниже

```
def __repr__(self):  
    """Строковое представление объекта"""  
    return (f"<Prediction(id={self.id}, filename='{self.filename}',"  
            f" class='{self.predicted_class}',"  
            f" confidence={self.confidence:.2%})>")
```

В переменную __tablename__ записывается имя, которое затем таблица будет иметь в базе данных (рис. 7).

После создания моделей баз данных можно приступать к реализации миграций.

Для этого сначала создаются конфигурационные файлы. С этой целью в терминале необходимо выполнить команду:

```
alembic init <имя директории>,
```

где <имя директории> - это имя директории в которой будут храниться версии миграций и конфигурационный файл именуемый env.py. Обычно это директорию называют «alembic». Таким образом, в терминале необходимо выполнить команду:

alembic init alembic

После ее выполнения в структуре проекта появится директория «alembic» внутри которой необходимо найти и открыть файл **env.py**. В данном файле необходимо добавить две строчки кода как показано на рисунке 4, а именно:

- необходимо из предварительно созданной модели импортировать класс **Base**. Для того чтобы применились миграции важно импортировать его именно из файла, в котором создавались таблицы базы данных при помощи соответствующих классов, а не из файла с подключением к БД. *То есть импортировать класс, который находится именно в sqlalchemy.orm*. В нашем случае, требуемый класс находится в пакете “db_model”, в файле “result.py” (рис. 3);

- затем необходимо инициализировать переменную target_metadata метаданными из импортированного класса Base.

```

# Interpret the config file for Python logging.
# This line sets up loggers basically.
if config.config_file_name is not None:
    fileConfig(config.config_file_name)

# add your model's MetaData object here
# for 'autogenerate' support
# from myapp import mymodel
# target_metadata = mymodel.Base.metadata
from db_model.results import Base
target_metadata = Base.metadata

```

Рисунок 4. – Инициализация переменной target_metadata в файле env.py

Следующий этап настройки выполняется в файле alembic.ini. Этот файл тоже должен появиться в проекте. Найдите его и откройте. В данном файле необходимо найти переменную sqlalchemy.url. По умолчанию там будет прописана строка соединения с базой данных, то есть с файлом, в котором должна находиться база данных (в данном примере этот файл назван predict.db). Это файл должен быть с расширением db. Его название и местоположение в проекте должны корреспондироваться с названием и указанным местоположением в файле с подключением к базе данных (рисунок 2). В рассматриваемом примере строка соединения будет выглядеть как на рисунке 5. В данном задании сделайте также, то есть исправьте строку подключения с той, что находится по умолчанию на sqlite:///predict.db

```

# the output encoding used when revision files
# are written from script.py.mako
# output_encoding = utf-8

# database URL. This is consumed by the user-maintained env.py script only
# other means of configuring database URLs may be customized within the env
# file.
sqlalchemy.url = sqlite:///predict.db

[post_write_hooks]
# post_write_hooks defines scripts or Python functions that are run
# on newly generated revision scripts. See the documentation for further
# detail and examples

```

Рисунок 5. – Соединение с базой данных в файле «alembic.ini»

Теперь наша модель базы данных, реализованная с помощью библиотеки SQLAlchemy готова к первой миграции. В результате применения миграции модели-сущности, реализованные при помощи Python классов будут конвертированы в

соответствующие SQL таблицы. Миграция выполняется двумя командами в терминале. Сначала выполните в терминале команду:

alembic revision --autogenerate -m "initial migration"

флаг -m "initial migration" – опциональный, но его обычно используют чтобы как-то обозначать, поименовать версии миграции (версии баз данных)

Затем, чтобы миграции применились необходимо выполнить команду

alembic upgrade head

В директории "alembic" проекта, в поддиректории «versions» должна появиться первая версия миграции, примерно, как показано на рисунке 6.

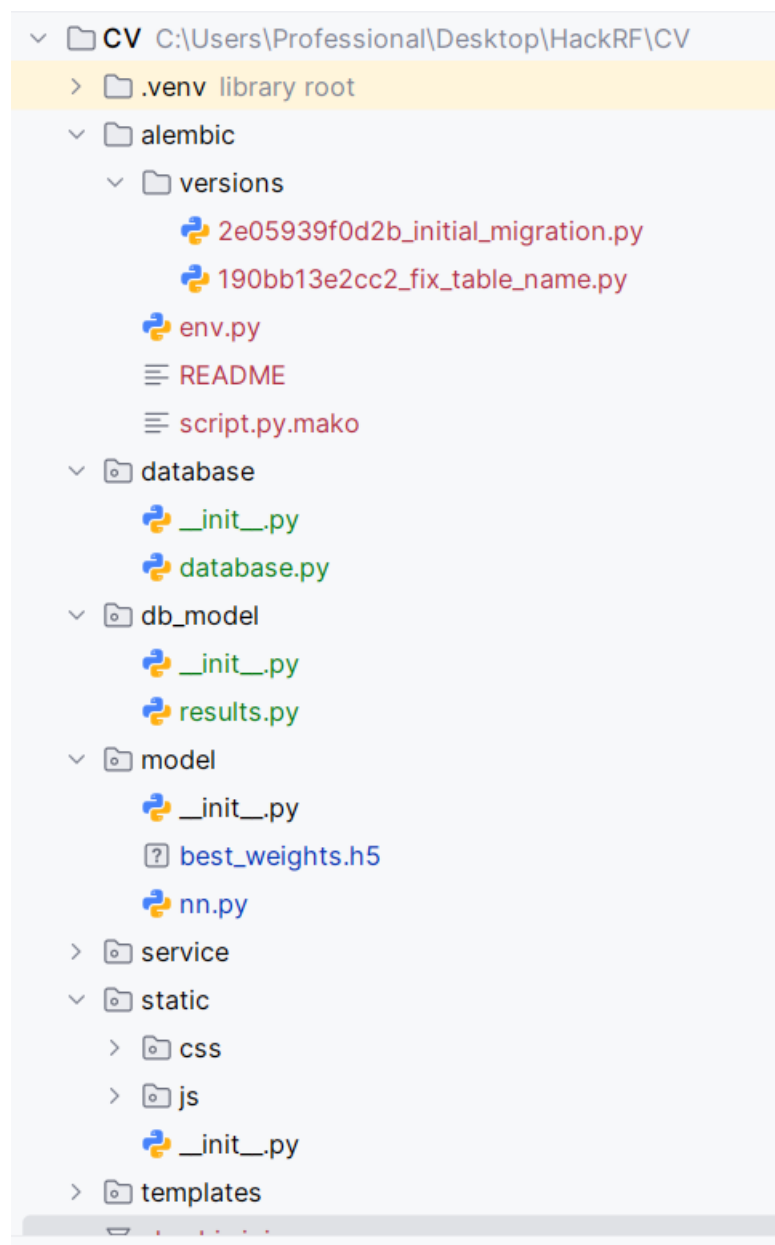


Рисунок 6. – файл с первой версией миграции

Также в структуре проекта должен появиться файл с базой данных. В нашем случае он называется predict.db

Его можно открыть используя, например СУБД DB Browser (SQLite). После открытия файла с базой данных в СУБД можно увидеть созданную базу данных (рисунок 7)

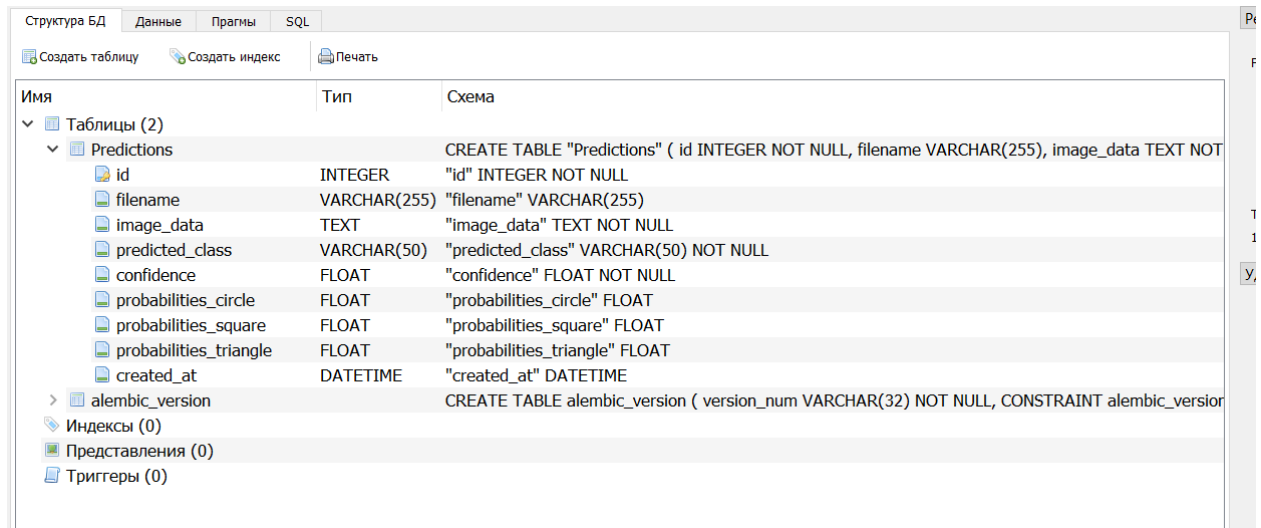


Рисунок 7. – База данных после применения первой миграции

Как видим, все атрибуты таблицы соответствуют переменным соответствующего класса в модели, которая была создана с помощью библиотеки SQLAlchemy (рис. 3).

Изображение для классификации геометрической фигуры будет отправляться с front-end при помощи POST запроса. Затем объект на изображении будет классифицирован при помощи предварительно обученной модели искусственной нейронной сети. Результаты классификации и само изображение должно быть сохранено в базе данных. Она уже создана. Сохранение осуществляется следующим образом: сначала создается объект класса Predictions, так мы назвали соответствующую модель приложения. Инициализируется объект данного класса при помощи полученного изображения, конвертированного в строку, и данных о предсказании, полученных с помощью модели искусственной нейронной сети. В контроллер необходимо сделать необходимые импорты, чтобы работать с моделью и базой данных. Нужно добавить две строчки кода:

```
from db_model.results import Predictions
from database.database import db_session, init_db
```

После чего модифицировать контроллер добавив инициализацию объекта и добавление его в базу данных, то есть добавить код:

```
# Конвертируем изображение в base64 для сохранения
image_base64 = base64.b64encode(image_bytes).decode('utf-8')

# Создаём запись в БД
new_prediction = Predictions(
    filename=file.filename,
    image_data=image_base64,
    predicted_class=predicted_class,
    confidence=confidence,
    probabilities_circle=float(predictions[0]), # круг
```

```

        probabilities_square=float(predictions[2]), # квадрат
        probabilities_triangle=float(predictions[1]) # треугольник
    )

    # Сохраняем в базу
    db_session.add(new_prediction)
    db_session.commit()

```

Этот код необходимо добавить в метод эндпоинта который вызывается при старте приложения. Собственно, пока в проекте нет других эндпоинтов. После модификации контроллера его код будет выглядеть следующим образом:

```

import base64
from flask import Flask, render_template, request, jsonify
from model import nn
import numpy as np
import io
from PIL import Image
from db_model.results import Predictions
from database.database import db_session, init_db

app = Flask(__name__)

# Инициализируем базу данных
init_db()
# Загружаем модель при старте
model = nn.get_model()

@app.route('/', methods=['GET', 'POST'])
def index():
    if request.method == 'POST':
        try:
            # Проверяем, есть ли файл в запросе
            if 'file' not in request.files:
                return jsonify({'error': 'Файл не найден'}), 400

            file = request.files['file']

            # Проверяем, выбран ли файл
            if file.filename == '':
                return jsonify({'error': 'Файл не выбран'}), 400

            # Читаем файл
            image_bytes = file.read()

            # Открываем изображение
            img = Image.open(io.BytesIO(image_bytes)).convert('L')
            img = img.resize((20, 20))

            # Предобработка
            img_array = np.array(img) / 255.0
            #img_array = 1.0 - (np.array(img) / 255.0) # Инвертированные
            цвета если необходимо
            img_flattened = img_array.flatten().reshape(1, -1)

            # Предсказание
            predictions = model.predict(img_flattened, verbose=0)[0]

            # Классы
            classes = ['circle', 'triangle', 'square']
            class_index = np.argmax(predictions)
            confidence = float(predictions[class_index])

```



```

predicted_class = classes[class_index]

print(predictions)

#СОХРАНЯЕМ В БАЗУ ДАННЫХ
# Конвертируем изображение в base64 для сохранения
image_base64 = base64.b64encode(image_bytes).decode('utf-8')

# Создаём запись в БД
new_prediction = Predictions(
    filename=file.filename,
    image_data=image_base64,
    predicted_class=predicted_class,
    confidence=confidence,
    probabilities_circle=float(predictions[0]), # круг
    probabilities_square=float(predictions[2]), # квадрат
    probabilities_triangle=float(predictions[1]) # треугольник
)

# Сохраняем в базу
db_session.add(new_prediction)
db_session.commit()

# Возвращаем JSON для JavaScript
return jsonify({
    'class': predicted_class,
    'confidence': confidence
})

except Exception as e:
    return jsonify({'error': str(e)}), 500

# GET запрос - показываем страницу
return render_template('index.html')

if __name__ == '__main__':
    app.run(debug=True)

```

Протестируйте приложение. Если все работает корректно, то по результатам классификации в базу данных запишутся ее результаты. Их можно посмотреть в системе управления базой данных. Один из примеров как она может выглядеть приведен на рисунке 8.

Новая база данных Открыть базу данных Записать изменения Отменить изменения Открыть проект Сохранить проект Прикрепить БД Закрыть								
Структура БД Данные Правки SQL								
Таблица: Predictions Filter in any column								
id	filename	image_data	predicted_class	confidence	probabilities_circle	probabilities_square	probabilities_triangle	probabilities_circle
Фи...	Фильтр	Фильтр	Фильтр	Фильтр	Фильтр	Фильтр	Фильтр	Фильтр
1	1 Cirl.png	iVBORw0KGgoAAAANSU...	circle	0.889162003993988	0.889162003993988	0.103024899959564	0.0078	0.0078
2	2 tr2.png	iVBORw0KGgoAAAANSU...	circle	0.847862124443054	0.847862124443054	0.129844024777412	0.022	0.022
3	3 Cirl.png	iVBORw0KGgoAAAANSU...	circle	0.889162003993988	0.889162003993988	0.103024899959564	0.0078	0.0078
4	4 Cirl.png	iVBORw0KGgoAAAANSU...	circle	0.889162003993988	0.889162003993988	0.103024899959564	0.0078	0.0078
5	5 Cirl.png	iVBORw0KGgoAAAANSU...	circle	0.889162003993988	0.889162003993988	0.103024899959564	0.0078	0.0078
6	6 Triangle.png	iVBORw0KGgoAAAANSU...	circle	0.868102729320526	0.868102729320526	0.0976669117808342	0.034	0.034

Рисунок 8. – Результат сохранения данных в базу данных